

SEN381 | SOFTWARE ENGINEERING 381

Quality, Security and Production Readiness

Assignment 3 | CivicConnect

PREPARED BY

Ryno Martin Goetz | 602306

Steven Riaan Piek | 602240

Willem Booysen | 600613

TEAM PARTICIPATION

All three listed team members contributed actively.

VERSION AND DATE

Version 1.0 | 21 September 2026

Contents

Contents.....	2
Executive overview	3
Team contribution and accountability	3
Question 1 Quality Strategy and Risk Based Verification	3
Quality assurance quality control verification and validation	3
Risk based verification	4
Complementary forms of verification evidence	4
Risk to Verification Evidence Map	5
Automation and quality gates	5
Critical judgement on a green pipeline.....	5
Question 2 Security Engineering and Threat Based Controls	6
Threat modelling with STRIDE.....	6
Threat to Control Traceability Table.....	6
How design construction and checking complement one another	7
Limitations of automated tools and AI recommendations.....	7
Critical judgement on a clean scanner result.....	8
Question 3 Production Readiness Deployment and Operational Evidence.....	8
Environment strategy and configuration parity	8
Secrets and production configuration	8
Release and deployment evidence	8
Failure recovery and data compatibility	9
Operations observability and alerts.....	9
Operational quality and trade offs	9
Production Readiness Evidence Matrix	9
Critical judgement on developer machine success	10
Question 4 Research to M3 Engineering Map	10
References.....	11
AI Usage Register.....	13

Executive overview

This brief researches how a modern web information system can build quality, security and production-readiness confidence from traceable evidence. The central finding is that no single test run, scanner result or successful deployment is sufficient. Risk should determine the depth and combination of evidence; threats should be translated into requirements, controls and checks; and production readiness should combine repeatable delivery with recovery, observability and operational ownership.

The research deliberately stops before project commitment. The final Research to M3 Engineering Map identifies candidate approaches and the decisions Milestone 3 must still make using CivicConnect's actual requirements, architecture, implementation, constraints and risks.

Team contribution and accountability

Member	Main contribution
Ryno Goetz	Question 1 quality strategy and risk-based verification; original Risk to Verification Evidence Map; A3-MAP-01 and A3-MAP-02
Steven Riaan Piek	Question 2 security engineering and threat-to-control traceability; A3-MAP-03 and A3-MAP-04
Willem Booysen	Question 3 production readiness and evidence matrix; A3-MAP-05 and A3-MAP-06; final integration

The assignment is jointly submitted, and all three members remain accountable for the complete document.

Question 1 Quality Strategy and Risk Based Verification

A defensible quality strategy connects stakeholder expectations and product risks to planned assurance activities, observable product evidence and explicit progression decisions. It therefore asks what can fail, who or what is harmed, how likely the failure is, what evidence would reveal it and what uncertainty remains. ISO/IEC 25010 supplies a quality model that can help teams express characteristics such as functional suitability, reliability, performance efficiency, security, maintainability and interaction capability as measurable objectives rather than vague claims (ISO and IEC, 2023).

Quality assurance quality control verification and validation

Quality assurance is process-oriented and preventive. It establishes the working conditions under which quality is likely to be built in, for example clear acceptance criteria, review rules, coding standards, test planning, traceability and controlled change. Quality control evaluates the resulting product or work product through inspection, measurement and testing. Verification asks whether the specified requirements and design were implemented correctly, while validation asks whether the resulting system is fit for stakeholder needs and its intended context. These activities overlap in practice: a review can verify a requirement, an automated test can control product quality, and stakeholder evaluation can validate whether technically correct behaviour solves the right problem.

The distinction matters because no single activity covers the whole quality problem. A team can follow a disciplined process yet misunderstand the need; it can pass implementation checks against an incomplete requirement; or it can satisfy a user scenario while concealing reliability or security weaknesses. ISO/IEC/IEEE 29119-2 treats testing as governed and managed processes across lifecycle models, supporting the view that evidence is planned, monitored and interpreted rather than appended at the end (ISO, IEC and IEEE, 2021).

Risk based verification

Risk-based testing selects, prioritises and manages test activities using risk analysis (ISTQB, 2024). Product risk combines the possibility of failure with its consequence. Business criticality adds context: failure in a rarely used administrative convenience may be tolerable, whereas a defect that exposes personal information, corrupts an audit trail or prevents recovery may justify deeper and more independent evidence even when its likelihood is uncertain.

A practical risk assessment can score likelihood and impact, then apply a criticality modifier for safety, security, legal, financial or mission consequences. The score is a prioritisation aid rather than an objective truth. The team should also record assumptions and confidence because little historical data can make a precise number misleading. Higher risk should normally increase the depth, diversity, timing and environment realism of verification: earlier reviews, more negative and boundary cases, independent review, integration with real dependencies, production-like load, and tighter exit criteria. Lower risk may justify sampling or lighter automation. Risk must be reassessed after design changes, defects, dependency updates or new operational evidence.

Complementary forms of verification evidence

Evidence form	Strong at exposing	Cannot prove on its own
Unit or component tests	Incorrect calculations, branching, state rules and boundary behaviour inside a small unit; fast localisation supports dense negative cases.	Correct collaboration, real database behaviour, deployment configuration, usability and end-to-end workflow.
Integration and API tests	Contract mismatches, serialisation, authentication or authorisation at interfaces, transaction boundaries, persistence mapping and dependency failure handling.	Browser-level experience, production scale, every external dependency condition and whether the requirement itself is right.
System and end-to-end tests	Broken user journeys, cross-component configuration, routing, session and workflow problems in a representative environment.	Exhaustive combinations, precise fault localisation, internal code quality or high-volume behaviour unless designed for it.
Static analysis and review	Suspicious code paths, unsafe constructs, style or maintainability issues, known patterns and defects visible without execution.	Runtime configuration, business-logic correctness, usable interaction, exploitability or absence of defects outside the rule set.
Performance and load evidence	Latency, throughput, saturation, capacity limits, resource leaks and failure behaviour under defined workloads.	Functional completeness, security, real future demand or production performance when the model and environment are unrepresentative.

The evidence forms are complementary because they trade speed and diagnostic precision against realism and breadth. A risk should be mapped to the smallest evidence bundle that can expose the credible failure modes. More tests are useful only when their oracles, data, environment and coverage correspond to those risks.

Risk to Verification Evidence Map

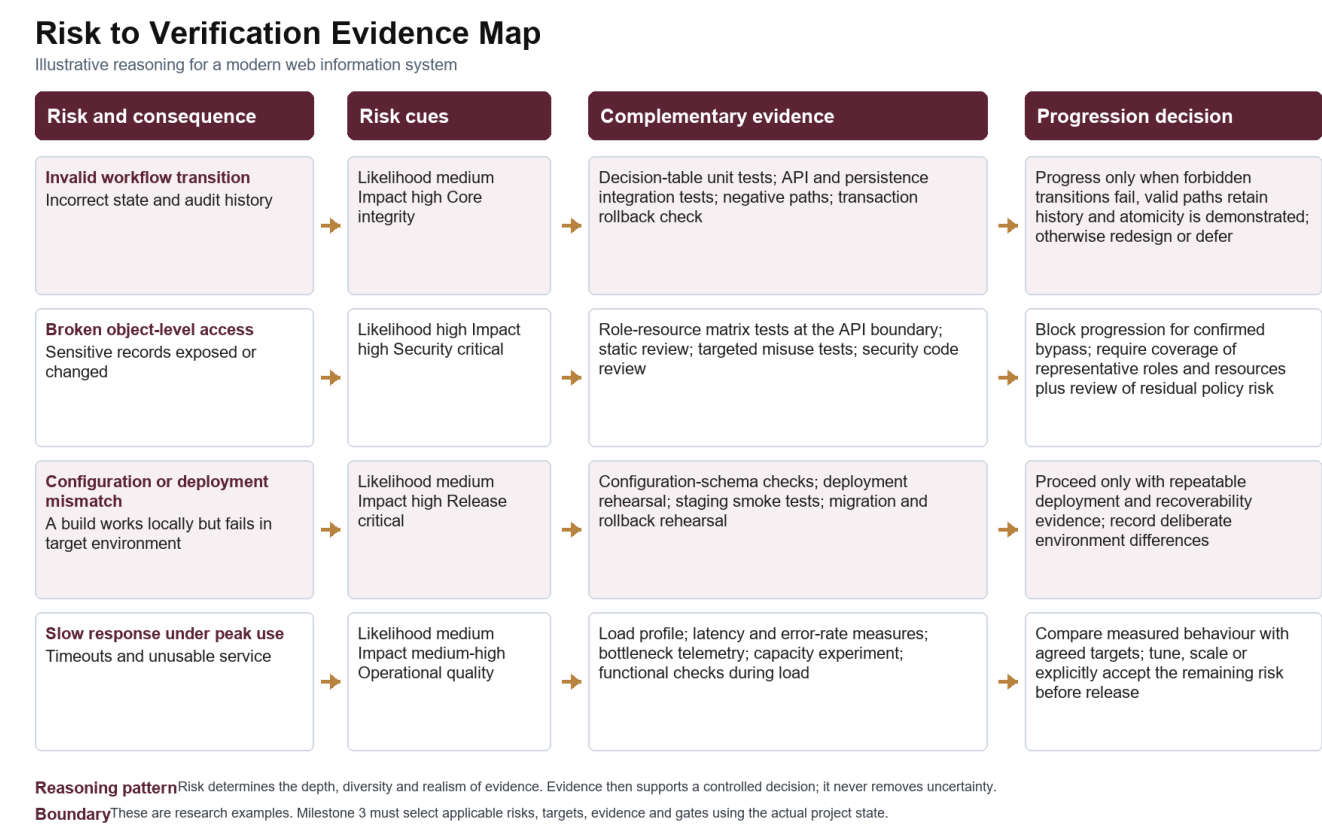


Figure 1 Original Risk to Verification Evidence Map

The map uses four illustrative risks to show the reasoning chain required by a quality strategy. It deliberately assigns evidence bundles rather than a single test type. For example, an access-control risk needs behavioural checks at the protected boundary plus design and code review; a deployment mismatch requires evidence from the deployment path and recovery process. The final column converts results into a progression decision and retains residual uncertainty.

Automation and quality gates

Automation makes selected checks repeatable, fast and visible. Candidate pipeline gates include compilation, static rules, unit and integration suites, dependency checks, test-result thresholds, migration validation and deploy-smoke checks. A useful gate has a stated risk purpose, an accountable owner, reliable execution, preserved evidence and an exception process. NIST's Secure Software Development Framework similarly treats automated tools as part of a broader set of practices that includes requirements, review, protected environments, provenance and vulnerability response (Scarfone, Souppaya and Dodson, 2022).

Poorly chosen gates create false confidence or delay work without reducing meaningful risk. Coverage percentages do not show that assertions are correct. Flaky checks encourage retries and normalise failure. A scanner can miss a novel or contextual defect, while a broad threshold can block low-risk findings and distract from a severe business-logic error. Gate criteria should therefore be reviewed when architecture, risk tolerance, dependencies or observed defects change.

Critical judgement on a green pipeline

All automated tests passing means only that the implemented checks produced their expected results in the environments and data used. It does not prove that every important requirement was

represented, that the assertions detect the right failures, that stakeholder needs were validated, that permissions and business rules are complete, or that the target environment will behave like the test environment. It also says little about unknown vulnerabilities, usability, recovery, operational ownership or load beyond the tested profile. Release confidence should therefore come from converging evidence linked to risk, together with explicit acceptance of what remains unproven.

Question 2 Security Engineering and Threat Based Controls

Security engineering converts credible misuse and threat scenarios into requirements, design and construction controls, verification evidence and residual-risk decisions throughout development. NIST's SSDF organises this work around preparing the organisation, protecting software, producing well-secured software and responding to vulnerabilities, while CISA's secure-by-design guidance places responsibility for secure defaults and systematic risk reduction into product engineering (Scarfone, Souppaya and Dodson, 2022; CISA et al., 2023). A final penetration test can discover some exposed weaknesses, but it cannot retroactively supply missing trust boundaries, requirements or accountability.

Threat modelling with STRIDE

Threat modelling is a structured analysis of how a design may be abused. A practical cycle defines the scope and security objectives, identifies assets and actors, draws data flows and trust boundaries, enumerates threats, assesses likelihood and impact, chooses a response, turns mitigations into traceable requirements, and verifies that the mitigations work. The model should be revisited when architecture, interfaces, data or dependencies change (Microsoft, 2026a).

STRIDE provides six prompts: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. Applying the prompts to data-flow elements makes discussion repeatable and accessible to non-specialists (Microsoft, 2026b). Its value is not the acronym itself; it helps the team ask focused questions such as whether an actor can impersonate another user, alter a request or record, deny a logged action, read protected data, exhaust a resource or gain stronger privilege. The resulting threat is then prioritised and connected to a measurable control objective.

STRIDE is a completeness aid rather than a risk calculator. Its findings depend on the accuracy of the model, the team's knowledge and the chosen scope. It may miss domain-specific abuse, chained attacks or flawed assumptions. Business stakeholders, developers and testers therefore need to review the model, and residual risk must remain visible rather than being marked solved because a category was considered.

Threat to Control Traceability Table

Threat or misuse	Control objective	Possible engineering control	Verification evidence	Residual risk or limitation
Authentication Credential stuffing, brute force or stolen session lets an attacker impersonate a user.	Establish risk-appropriate confidence in identity; resist automated attempts; protect authenticator and session lifecycle.	Candidate controls include approved password hashing, rate limiting, secure recovery, reauthentication for sensitive actions, protected cookies and risk-appropriate multifactor authentication.	Negative login tests; rate-limit and lockout checks; session fixation and expiry tests; recovery-path review; authentication event-log inspection.	Phishing, compromised endpoints and social engineering remain. Strong authentication does not prove authorisation. Assurance level must match context (NIST, 2025).
Authorisation An authenticated user changes an identifier or calls an endpoint to read or modify another	Deny by default and enforce least privilege and object-level permission on every protected request.	Central policy or middleware; server-side role and ownership checks; scoped service identities; explicit failure handling;	Role-resource-action matrix; API integration tests for horizontal and vertical escalation; manual route review; targeted	A correct mechanism can still implement a wrong policy. New endpoints, background jobs and administrative paths may

Threat or misuse	Control objective	Possible engineering control	Verification evidence	Residual risk or limitation
user's record.		protected audit events.	misuse tests.	escape coverage (OWASP Foundation, 2026a).
Input and sensitive data Malformed or crafted input triggers injection, mass assignment or unintended disclosure of personal information.	Accept only valid data, separate code from data, minimise sensitive fields and protect confidentiality through the data lifecycle.	Schema and allowlist validation; parameterised data access; output encoding; request-size limits; controlled field binding; encryption and minimised responses where justified.	Boundary and fuzz tests; injection tests; static review; API response inspection; data-flow and retention review; transport-configuration checks.	Validation cannot replace contextual output encoding or parameterisation. Tests may miss a new parser, field or business-logic disclosure (OWASP Foundation, 2026b).
Secrets and production configuration Credentials are committed, overprivileged, logged or reused, allowing unauthorised service or data access.	Keep secrets outside source and ordinary configuration; minimise scope and lifetime; support rotation, audit and revocation.	Managed secret store or protected injection mechanism; separate identities per environment; least privilege; masked logs; rotation and emergency revocation procedure.	Repository and pipeline secret scanning; access-policy review; rotation/revocation drill; configuration inspection; log-scrubbing tests.	Scanning may miss transformed or external leaks. A secure store cannot correct excessive privilege, poor rotation or compromised runtime access (OWASP Foundation, 2026c).

How design construction and checking complement one another

Secure design decides where trust changes, which assets need protection, which identities act, how failure is contained and which controls must exist. Secure coding implements those decisions through safe data access, validated boundaries, correct session and authorisation logic, careful error handling and protected logging. Dependency and security checks inspect known patterns, vulnerable components, leaked secrets and configuration weaknesses. OWASP ASVS 5.0 provides verifiable requirements across these areas, making it useful as a source of candidate checks rather than as proof that an application is secure (OWASP Foundation, 2025).

The activities are not substitutes. Static analysis cannot discover a missing business authorisation rule if the rule was never specified. A carefully reviewed design can still be undermined by an unsafe query or vulnerable library. Dependency scanning reports known component vulnerabilities but does not show whether a vulnerable path is reachable, whether a zero-day exists or whether an update breaks the product. The strongest evidence links a threat to a requirement, then to implementation and multiple checks. NIST CSF 2.0 similarly treats protection, detection, response and recovery as connected outcomes within risk governance (Pascoe, Quinn and Scarfone, 2024).

Limitations of automated tools and AI recommendations

- Coverage and context limits. Scanners recognise rules, signatures and reachable behaviours within their configuration. They commonly miss domain abuse, incorrect roles, flawed recovery journeys, unsafe operational assumptions and vulnerabilities outside scanned code or environments.
- False positives false negatives and configuration gaps. Results depend on rules, versions, build visibility, test data and exclusions. Teams may waste effort on noisy findings or overlook an excluded path, unsupported language, transitive component or runtime setting.
- Evidence interpretation. Severity labels are generic estimates. A medium technical finding may be critical in a sensitive workflow, while a nominally high issue may not be reachable. Human analysis is needed to determine exposure, consequence, treatment and acceptance.
- AI reliability and confidentiality. Generated advice can be outdated, fabricated or copied without system context; suggested controls may be insecure when implemented incorrectly. Sensitive architecture or data should not be exposed to an unapproved service. Every recommendation needs verification against current standards, official documentation, threat context and executable evidence.

Critical judgement on a clean scanner result

A report with no high-severity findings establishes only that the configured scanner did not identify such findings in the analysed scope and ruleset. It does not prove complete threat coverage, correct access policy, secure business logic, safe defaults, protected account recovery, runtime isolation, effective logging, dependency freshness, absence of unknown vulnerabilities or resilience after compromise. It also does not prove that findings labelled lower severity are harmless when combined. Security confidence therefore requires traceable requirements, design review, secure implementation, targeted negative and misuse tests, dependency and configuration evidence, and an explicit residual-risk decision.

Question 3 Production Readiness Deployment and Operational Evidence

Production readiness is an evidence-based judgement that a product can be deployed, operated, recovered and supported within defined risk and service expectations. Functional correctness is necessary but incomplete. Microsoft groups workload quality across reliability, security, cost optimisation, operational excellence and performance efficiency, emphasising that trade-offs must be decided against business requirements (Microsoft, 2025a). NIST CSF 2.0 also connects protection with detection, response and recovery rather than treating release as the end of engineering (Pascoe, Quinn and Scarfone, 2024).

Environment strategy and configuration parity

Development environments optimise fast feedback; test and staging environments provide controlled integration and release rehearsal; production serves real users and data under operational controls. Parity means important assumptions are intentionally aligned: the same build artefact and deployment mechanism, compatible runtime and data-store versions, an equivalent configuration schema, representative network and identity boundaries, and known differences recorded as risks. Exact equality is neither always possible nor desirable because lower environments should not contain production secrets or unrestricted personal data.

Uncontrolled differences create deployment risk because a test may exercise different dependencies, flags, permissions, migrations or resource limits from production. Configuration should be versioned where safe, validated against a schema and promoted through the same controlled path as code. Environment-specific values should be injected through controlled mechanisms, and a release should expose the effective non-secret configuration needed for diagnosis without revealing credentials.

Secrets and production configuration

Passwords, tokens, signing keys and connection credentials should be separated from source code and ordinary configuration. Candidate practices include a managed secrets service, narrow identities, per-environment values, short lifetimes where practical, rotation, revocation, audited access and masking in logs and pipeline output. OWASP emphasises central storage, provisioning, auditing and rotation, while NIST SSDF requires protected development environments and safeguarding of software and related artefacts (OWASP Foundation, 2026c; Scarfone, Souppaya and Dodson, 2022). The trade-off is operational complexity: a secret store that nobody can recover or rotate safely creates a different availability risk.

Release and deployment evidence

A successful build demonstrates that source was transformed and selected checks passed. A successful deployment additionally requires that an identified artefact reaches the intended environment, configuration and migrations are valid, services start, health checks reflect usable

behaviour, traffic reaches the release and a responsible person can observe the outcome. Repeatability comes from versioned deployment definitions, immutable or identifiable artefacts, controlled promotion, recorded approvals and automated smoke checks. Safe-deployment guidance recommends small quality-gated changes, health models, progressive exposure and immediate halt and recovery when issues appear (Microsoft, 2025b).

Automation reduces variation, but the release process still needs authorisation, segregation appropriate to risk, an exception path for emergencies and preserved evidence. Build and deployment pipelines are production systems themselves: their identities, dependencies and logs require protection. A release decision should state what evidence was checked, what deviations were accepted and who owns monitoring after deployment.

Failure recovery and data compatibility

Recovery planning starts before release. The team needs to know which failures can be reversed, which require a forward fix, how long detection and restoration may take, and how much data loss is tolerable. Rollback is credible only when the previous artefact and configuration are available and the data schema remains compatible. Destructive database changes may make an application rollback impossible; expand-and-contract migrations, backward-compatible reads and writes, staged data conversion and verified backups reduce this coupling.

A backup file is evidence that data was copied, not that service can be recovered. Restore tests should verify integrity, permissions, dependencies and the time to usable service. Recovery objectives, incident roles, communication and decision authority should match business impact. Exercises may reveal that a nominal recovery procedure depends on one unavailable person, undocumented credentials or obsolete infrastructure.

Operations observability and alerts

Logs record discrete events and context; metrics aggregate numeric behaviour over time; traces connect work across components and show the path and latency of a request. OpenTelemetry describes these as complementary signals rather than interchangeable records (OpenTelemetry Authors, 2026). Security logs require careful design because too little data hides misuse while excessive or sensitive logging raises cost and privacy risk (OWASP Foundation, 2026d).

Monitoring turns selected signals into views of service health, while alerts should identify an actionable condition with an owner, urgency and response. Google's SRE guidance recommends collecting metrics for a defined purpose such as alerting, diagnosis, capacity planning or trends and testing the monitoring system itself (Beyer et al., 2018). Telemetry without interpretation can create storage cost, noise and false confidence. Operational evidence should therefore connect user-relevant indicators, thresholds and changes to a runbook and a decision.

Operational quality and trade offs

Reliability concerns continuity and correct service, performance concerns response under a specified workload, scalability concerns behaviour as demand changes, cost concerns whether the operating model is sustainable, and supportability concerns whether people can diagnose and restore the system. These characteristics interact. Redundancy can improve availability but raises cost and operational complexity; aggressive caching can improve latency but introduce staleness; extensive telemetry can improve diagnosis while increasing expense and privacy exposure. Readiness criteria must therefore use measurable workload and recovery assumptions rather than universal targets.

Production Readiness Evidence Matrix

Readiness concern	Risk if ignored	Evidence before release	Evidence after release
Environment and	Untested differences cause	Versioned configuration schema;	Configuration drift; failed starts;

Readiness concern	Risk if ignored	Evidence before release	Evidence after release
configuration parity	startup, connectivity, permission or behaviour failures.	recorded differences; same artefact and deployment path; staging smoke result.	feature-flag state; environment-specific error rate.
Secrets and service identities	Credential leakage or excessive privilege enables unauthorised access and makes incidents hard to contain.	No repository secrets; scoped identities; protected injection; access review; rotation and revocation rehearsal.	Secret-access audit events; expiry and rotation failures; anomalous identity use; leakage alerts.
Repeatable release and deployment	Manual variance deploys the wrong artefact or produces an unrepeatable state.	Identified artefact; versioned deployment definition; approvals; health and smoke checks; release record.	Deployment success and duration; change-failure rate; health during rollout; rollback or fix-forward events.
Schema and data compatibility	Application and database versions conflict, corrupt data or prevent rollback.	Migration review; compatibility test; representative rehearsal; backup; rollback or forward-fix plan.	Migration errors; integrity checks; failed writes; old and new version compatibility during rollout.
Failure recovery and restore	An incident causes extended outage or unrecoverable data loss.	Defined recovery objectives; tested restore; incident contacts; accessible artefacts and runbook; exercise evidence.	Detection and restoration time; data-loss window; backup success and restore sampling; incident actions.
Observability and actionable alerts	Users fail while the team lacks evidence or receives unusable noise.	Structured logs; defined service indicators; dashboards; alert routing and runbook; monitoring test.	User-visible error and latency; alert precision; missing telemetry; response and resolution time.
Performance scalability and capacity	Load causes saturation, timeouts or uncontrolled cost.	Representative load model; baseline latency throughput and resources; capacity assumption; bottleneck analysis.	Demand, saturation and tail latency; scaling events; cost per workload unit; capacity forecast.
Ownership support and cost	Incidents remain unresolved, maintenance stalls or operating cost becomes unsustainable.	Named owner; support hours; escalation path; dependency inventory; cost estimate; known limitations.	Support load; unresolved alerts; dependency end-of-life; actual cost; recurring manual effort and toil.

Critical judgement on developer machine success

Software can pass functional tests and run locally while remaining unready because local evidence rarely represents production identity, configuration, data volume, concurrency, network failure, dependency behaviour, secret handling, migration compatibility, monitoring, backup restoration or support arrangements. The deployment path may never have been exercised and the team may be unable to detect or reverse a bad release. Production readiness therefore requires controlled pre-release evidence and continuing operational evidence, with explicit ownership of the remaining risk.

Question 4 Research to M3 Engineering Map

The map transfers the strongest research findings into candidate evidence and open Milestone 3 decisions. The final column preserves the boundary: research expands the decision space, while M3 must use the actual CivicConnect product state to select, apply and verify controls.

Research finding	Engineering concern	Candidate approach or evidence	Trade off or limitation	Decision M3 must still make
Risk-based verification allocates depth and diversity of evidence according to likelihood, impact and criticality.	Unequal consequence of product failures	Risk register linked to unit, integration, system, performance and review evidence with explicit progression criteria.	Scores are uncertain and can become stale; more testing does not guarantee useful coverage.	Which CivicConnect risks justify deeper or independent evidence, and what thresholds support progression?

Research finding	Engineering concern	Candidate approach or evidence	Trade off or limitation	Decision M3 must still make
A green automated pipeline is evidence about implemented checks, not complete product quality.	False confidence from automation	Layered evidence, reviewed test oracles, environment-aware gates and controlled exceptions.	Manual evidence costs time; broad gates can be noisy or block low-risk work.	Which checks should become gates, which require human review, and how will exceptions be approved?
Threat modelling links credible misuse to requirements, design responses and tests before release.	Security gaps caused by missing context or trust boundaries	Versioned data-flow and trust-boundary model, STRIDE prompts, prioritised threats and traceable verification.	Completeness depends on model accuracy and team knowledge; STRIDE can miss domain abuse.	Which CivicConnect flows, assets and boundaries require modelling, and who accepts residual threats?
Automated security and dependency checks need contextual review and targeted misuse evidence.	Unknown, business-logic and supply-chain weaknesses	Candidate SAST, SCA and secret checks combined with access-matrix tests, code review and verification against ASVS.	False results, unsupported paths and new vulnerabilities remain; tool output needs interpretation.	Which tools and ASVS requirements fit the stack, and what severity or context should block progression?
Repeatable deployment must include environment compatibility, health evidence and a credible recovery path.	Release failure and unrecoverable change	Versioned deployment path, configuration validation, staging rehearsal, compatible migrations and rollback or fix-forward exercise.	Parity has cost; rollback may be impossible after destructive data changes.	What target environment, migration method and recovery evidence are proportionate to CivicConnect?
Observability is useful when signals answer service questions and alerts lead to owned action.	Invisible failure and slow incident response	Candidate logs, metrics and traces mapped to user-relevant indicators, thresholds, dashboards and runbooks.	Telemetry can be expensive, noisy and privacy-sensitive; thresholds require operational learning.	Which service indicators, alerts, retention and ownership are needed for the actual operating model?

References

All sources were published or actively maintained between 2018 and 2026 and fall within the required ten-year window.

International Organization for Standardization and International Electrotechnical Commission (2023) ISO/IEC 25010:2023 Systems and software engineering - Systems and software Quality Requirements and Evaluation - Product quality model. Geneva: ISO. Available at: <https://www.iso.org/standard/78176.html> (Accessed: 21 September 2026).

International Organization for Standardization, International Electrotechnical Commission and Institute of Electrical and Electronics Engineers (2021) ISO/IEC/IEEE 29119-2:2021 Software and systems engineering - Software testing - Part 2: Test processes. Geneva: ISO. Available at: <https://www.iso.org/standard/79428.html> (Accessed: 21 September 2026).

International Software Testing Qualifications Board (2024) Certified Tester Foundation Level Syllabus v4.0.1. Available at: https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf (Accessed: 21 September 2026).

Scarfone, K., Souppaya, M. and Dodson, D. (2022) Secure Software Development Framework Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218. Gaithersburg, MD: National Institute of Standards and Technology. Available at: <https://doi.org/10.6028/NIST.SP.800-218> (Accessed: 21 September 2026).

Pascoe, C., Quinn, S. and Scarfone, K. (2024) The NIST Cybersecurity Framework 2.0. NIST CSWP 29. Gaithersburg, MD: National Institute of Standards and Technology. Available at: <https://doi.org/10.6028/NIST.CSWP.29> (Accessed: 21 September 2026).

National Institute of Standards and Technology (2025) Digital Identity Guidelines. NIST SP 800-63-4. Gaithersburg, MD: NIST. Available at: <https://pages.nist.gov/800-63-4/> (Accessed: 21 September 2026).

OWASP Foundation (2025) OWASP Application Security Verification Standard 5.0.0. Available at: <https://owasp.org/projects/asvs> (Accessed: 21 September 2026).

Cybersecurity and Infrastructure Security Agency, National Security Agency and international partners (2023) Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Security-by-Design and Default. Available at: https://www.cisa.gov/sites/default/files/2023-10/SecureByDesign_1025_508c.pdf (Accessed: 21 September 2026).

Microsoft (2026a) Microsoft Security Development Lifecycle. Available at: <https://learn.microsoft.com/en-us/compliance/assurance/assurance-microsoft-security-development-lifecycle> (Accessed: 21 September 2026).

Microsoft (2026b) Threats in the Microsoft Threat Modeling Tool. Available at: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats> (Accessed: 21 September 2026).

OWASP Foundation (2026a) Authorization Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 21 September 2026).

OWASP Foundation (2026b) Input Validation Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html (Accessed: 21 September 2026).

OWASP Foundation (2026c) Secrets Management Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html (Accessed: 21 September 2026).

OWASP Foundation (2026d) Logging Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html (Accessed: 21 September 2026).

Microsoft (2025a) Operational Excellence design principles. Azure Well-Architected Framework. Available at: <https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/principles> (Accessed: 21 September 2026).

Microsoft (2025b) Architecture strategies for safe deployment practices. Azure Well-Architected Framework. Available at: <https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/safe-deployments> (Accessed: 21 September 2026).

Beyer, B., Murphy, N.R., Rensin, D.K., Kawahara, K. and Thorne, S. (eds.) (2018) The Site Reliability Workbook. Sebastopol, CA: O'Reilly Media. Available at: <https://sre.google/workbook/table-of-contents/> (Accessed: 21 September 2026).

OpenTelemetry Authors (2026) Observability primer. Available at: <https://opentelemetry.io/docs/concepts/observability-primer/> (Accessed: 21 September 2026).

AI Usage Register

AI assisted with research and drafting. It was not used as an academic source; each member checked the material against the cited standards and official guidance.

Date	Student	Tool	Engineering task	AI contribution	Human verification	Decision	Issues found
21 September 2026	Ryno Goetz	OpenAI Codex	Quality strategy, risk-based verification and the Risk to Verification Evidence Map	Source discovery, synthesis, comparative analysis, drafting and visual layout	Compared claims with ISO/IEC 25010:2023, ISO/IEC/IEEE 29119-2:2021, ISTQB CTFL 4.0.1 and NIST SSDF 1.1	Accepted with corrections	Removed older sources, a generic testing list and wording that treated a green pipeline as complete quality evidence
21 September 2026	Steven Riaan Piek	ChatGPT	Security engineering, STRIDE threat modelling and threat-to-control traceability	Source discovery, synthesis of security guidance, drafting the four concern chains and citation checks	Compared chains with NIST SSDF, NIST SP 800-63-4, NIST CSF 2.0, OWASP ASVS 5.0 and current Microsoft and OWASP guidance	Accepted with corrections	Removed claims that scanners prove security, separated authentication from authorisation and retained residual risks
21 September 2026	Willem Booysen	Microsoft Copilot	Production readiness, deployment, recovery, observability and the evidence matrix	Source discovery, research synthesis, matrix drafting and consistency checks across operational concerns	Compared claims with NIST CSF 2.0, NIST SSDF, Azure Well-Architected guidance, Google SRE, OpenTelemetry and OWASP guidance	Accepted with corrections	Removed the assumption that successful deployment proves readiness and added recovery, post-release evidence and operational trade-offs